COMPLETE PROJECT DOCUMENTATION

CleanCity AI

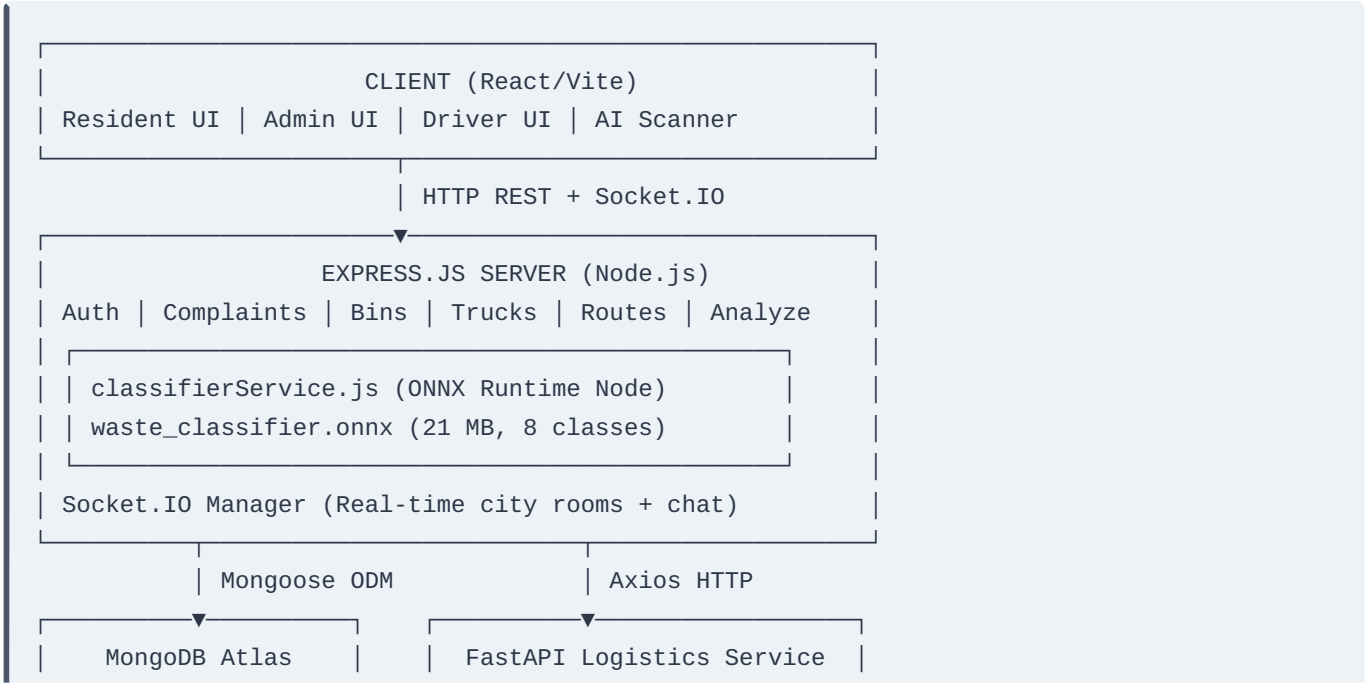
1. Project Overview

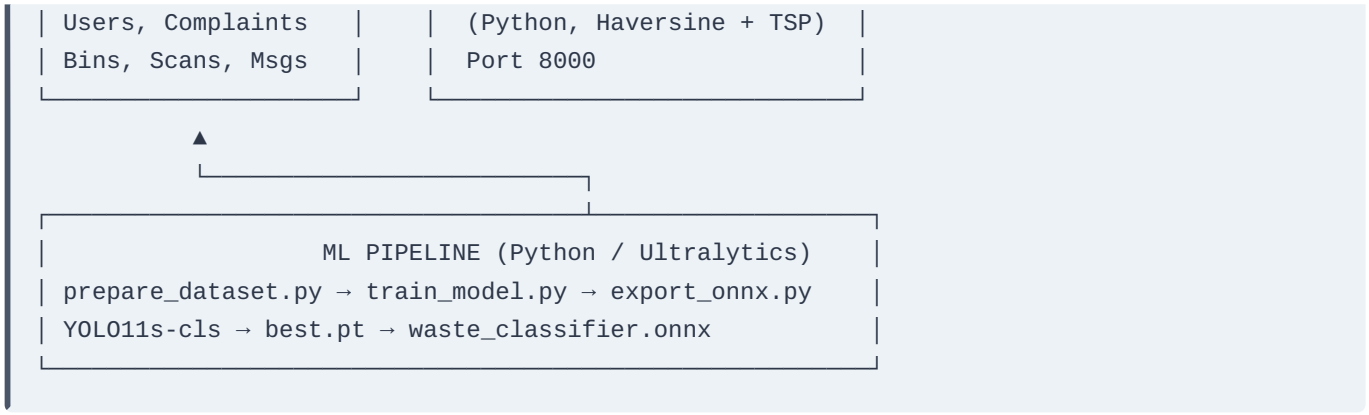
CleanCity AI is a full-stack smart waste management platform that combines computer vision, real-time logistics, and role-based dashboards to transform urban waste collection. Citizens scan waste items to get AI-powered disposal instructions, report overflowing bins, and track collectors in real time — just like an Uber for garbage collection.

Core Value Proposition

Problem	CleanCity AI Solution
Citizens don't know how to sort waste	AI scanner identifies material & bin type instantly
Overflowing bins go unreported	Residents submit geo-tagged complaints
Inefficient collection routes	TSP-optimized driver routing via FastAPI
No accountability in collection	Full lifecycle: report → assign → track → reward
No incentive for participation	EcoCredits for residents, Earnings for drivers

2. System Architecture





3. Technology Stack

3.1 Frontend (Client)

Technology	Version	Purpose
React	19.2	Component-based UI framework
Vite	7.x	Build tool & dev server
React Router DOM	7.x	Client-side routing & protected routes
Tailwind CSS	3.4	Utility-first styling
Framer Motion	12.x	Animations & micro-interactions
Socket.IO Client	4.8	Real-time WebSocket communication
Axios	1.x	HTTP API client
Recharts	3.x	Analytics charts & graphs
React Webcam	7.x	Live camera feed for waste scanner
Lucide React	0.56	Icon library
@react-google-maps/api	2.x	Interactive map components
@tensorflow/tfjs	4.x	Client-side TF (auxiliary)

3.2 Backend (Server)

Technology	Version	Purpose
Node.js	LTS	JavaScript runtime

Technology	Version	Purpose
Express.js	4.x	REST API framework
Socket.IO	4.8	Real-time bidirectional events
Mongoose	8.x	MongoDB ODM & schema validation
JSON Web Token (JWT)	9.x	Stateless authentication
bcryptjs	2.x	Password hashing
Multer	1.x	File upload middleware
Sharp	0.33	Image processing & resizing
ONNX Runtime Node	1.17	On-device AI inference
Axios	1.x	HTTP client (calls FastAPI)
dotenv	16.x	Environment variable management

3.3 ML Pipeline

Technology	Purpose
Python 3.x	ML scripting language
Ultralytics YOLO	YOLO11s-cls model training
PyTorch	Deep learning backend
ONNX / onnxruntime	Model export & CPU inference
Albumentations	Image augmentation library
Pillow (PIL)	Image loading & conversion
NumPy	Tensor/array operations
tqdm	Progress bars

3.4 Logistics Service

Technology	Purpose
FastAPI	Async Python web framework

Technology	Purpose
Pydantic	Request/response data validation
NumPy	Haversine distance calculations
Uvicorn	ASGI server

3.5 Database

Technology	Purpose
MongoDB	NoSQL document database
MongoDB Atlas	Cloud-hosted cluster
Mongoose 2dsphere Index	Geospatial queries on user locations

4. User Roles

4.1 Resident

- Register/login with city selection
- Use AI waste scanner (camera or file upload)
- Submit bin overflow complaints with GPS location
- Track assigned driver on live map
- Chat with assigned driver
- View EcoCredits balance and scan history
- View personal analytics

4.2 Admin

- View full city dashboard with KPIs
- Manage all bins (CRUD, fill-level monitoring)
- Review and triage complaints (ping drivers / decline)
- Finalize resolved reports and authorize credits
- Manage truck fleet
- Manage all users
- View city-wide analytics

4.3 Driver

- View city complaints broadcast to their area

- Accept or skip available jobs
- Send live GPS heartbeat to resident's map
- Chat with resident during job
- Submit waste summary upon completion
- View earnings history

5. Feature-by-Feature Explanation

5.1 AI Waste Scanner

Location: `client/src/pages/resident/WasteScanner.jsx`, `components/scanner/`

The scanner lets residents identify waste material by either taking a live camera photo or uploading an image file.

Flow:

1. User opens camera (`CameraFeed.jsx`) or drops a file (`DropZone.jsx`)
2. Image is sent as `multipart/form-data` to `POST /api/analyze`
3. Server saves file via Multer → reads buffer → calls `classifierService.classify()`
4. ONNX model runs inference → softmax → returns material + bin category
5. Result displayed in `ResultCard.jsx` with confidence bar, category breakdown, disposal instructions

Output fields:

- `material` — specific material (Plastic, Metal, Glass, Paper, Organic, Electronic, Medical, Textile)
- `binCategory` — disposal bin (Recyclable, Compostable, Hazardous, General)
- `materialConfidence` — float 0–1
- `binInstruction` — human-readable disposal guidance
- `processingTimeMs` — inference latency

5.2 Complaint / Report System (Uber-Style)

Location: `server/controllers/complaintController.js`, `server/routes/complaintRoutes.js`

Complete lifecycle from report to reward:

```
Resident submits report
        ↓
Admin pings city drivers (broadcast via Socket.IO)
        ↓
Driver accepts job (status: on-the-way)
        ↓
Driver sends GPS heartbeat → Resident sees live map
        ↓
```

```
Driver resolves (submits waste summary)
        ↓
Admin finalizes + writes remarks → Rewards dispersed
```

- **Status States:** `pending` → `on-the-way` → `resolved` → (Admin) → `finalized`
- **Also:** Admin can `decline` (status: `rejected`)
- **Reward Logic (finalizeReport):**
  - Authorized collection: Driver +₹50, Resident +25 EcoCredits
  - Unauthorized: Driver +₹10, Resident +5 EcoCredits

5.3 Real-Time Communication (Socket.IO)

Location: `server/services/socketManager.js`

Rooms

Room Type	Join Event	Purpose
City Room	`join_city_room`	Broadcast jobs to all drivers in a city
Personal Room	`join_personal_room`	Notify specific resident
Chat Room	`join_chat_room`	Resident ↔ Driver messaging

Events

Event	Direction	Meaning
`new_report_submitted`	Server→Drivers	New complaint in city
`job_broadcast_alert`	Server→Drivers	Admin pinged drivers
`driver_assigned`	Server→Resident	Driver accepted the job
`driver_location_update`	Server→Resident	Live GPS position
`job_claimed`	Server→Drivers	Job taken, stop showing it
`collection_completed`	Server→Resident	Driver finished
`report_declined`	Server→Resident	Admin rejected complaint
`chat_message`	Client→Server	Bidirectional chat message
`new_message`	Server→Both	Broadcast chat message

5.4 Route Optimization (Logistics)

Location: `server/logistics_service.py`, `server/routes/logisticsRoutes.js`

Algorithm: Nearest Neighbor Heuristic (TSP Approximation)

1. Driver requests `/api/logistics/optimize` with their `startLocation`
2. Node.js fetches all `full` / `overflowing` bins + `pending` complaints in the city
3. Combines into a list of `spots` (lat/lng)
4. Calls FastAPI `POST /optimize` with spot list
5. FastAPI runs Nearest Neighbor TSP:
   - Start at driver's current location
   - Repeatedly pick the closest unvisited spot
   - Uses **Haversine formula** for accurate Earth-surface distances
6. Returns ordered indices → Node.js maps back to full spot objects
7. Driver app renders the optimized route on map

Haversine Formula:

$$d = 2R \times \arctan2(\sqrt{a}, \sqrt{(1-a)})$$

$$a = \sin^2(\Delta lat/2) + \cos(lat1) \cdot \cos(lat2) \cdot \sin^2(\Delta lng/2)$$

5.5 Bin Management

Location: `server/controllers/binController.js`, `server/models/binModel.js`

Admins manage physical waste bins across the city:

- **Types:** General Waste, Recycling, Organic, Hazardous
- **Statuses:** `empty` → `half-full` → `full` → `overflowing`
- **Fill Level:** 0–100% percentage
- **Auto-update:** When a resident reports a bin, its status auto-sets to `overflowing` (fill=100)
- City filter ensures admins only see their city's bins

5.6 Authentication & Authorization

Location: `server/controllers/authController.js`, `server/middleware/authMiddleware.js`

Method: JWT Bearer Token (30-day expiry)

Middleware Guards:

- `protect` — verifies JWT, attaches `req.user`
- `admin` — checks `req.user.role === 'admin'`
- `driver` — checks role is `driver` or `admin`

Password Security: bcryptjs with salt rounds = 10, pre-save Mongoose hook

5.7 Admin Dashboard

Location: `client/src/pages/admin/`

Page	Purpose
`AdminDashboard.jsx`	KPI summary: total complaints, bins, trucks, users
`AdminComplaints.jsx`	Full complaint table with ping/decline/finalize actions
`AdminBins.jsx`	Bin CRUD with map overlay and fill-level badges
`AdminTrucks.jsx`	Truck fleet management
`AdminUsers.jsx`	User listing, role management
`AdminAnalytics.jsx`	Charts: complaint trends, resolution rates

5.8 Driver Dashboard

Location: `client/src/pages/driver/DriverDashboard.jsx`

- Shows real-time broadcast jobs from Socket.IO `job_broadcast_alert`
- Accept/skip individual jobs
- Active job map with live resident location
- GPS heartbeat sender to update `driverLocation` in DB
- In-job chat with resident
- Submit waste summary form on completion

5.9 Resident Dashboard

Location: `client/src/pages/resident/ResidentDashboard.jsx`

- Live map showing all bins in city (color-coded by status)
- Submit new complaint form with location picker
- Active job tracker with driver position updates
- EcoCredits balance display
- Quick-access AI scanner
- Notification feed from Socket.IO events

6. ML Pipeline Deep Dive

6.1 Dataset Preparation (`prepare_dataset.py`)

Sources Merged:

- **TrashBox** dataset — plastic, metal, glass, paper, cardboard, e-waste, medical
- **NewDataset** — plastic, metal, glass, paper, organic, e-waste, batteries, light bulbs, clothes

8-Class Unified Mapping

Class	Bin Category	Source Folders
Plastic	Recyclable	TrashBox/plastic + NewDataset/plastic
Metal	Recyclable	TrashBox/metal + NewDataset/metal
Glass	Recyclable	TrashBox/glass + NewDataset/glass
Paper	Recyclable	TrashBox/paper + TrashBox/cardboard + NewDataset/paper
Organic	Compostable	NewDataset/organic
Electronic	Hazardous	TrashBox/e-waste + NewDataset/e-waste + batteries + light bulbs
Medical	Hazardous	TrashBox/medical
Textile	General	NewDataset/clothes

Split: 70% Train / 20% Validation / 10% Test (fixed seed=42)

Augmentation Pipeline (Albumentations) for class balancing:

- Horizontal flip (p=0.5)
- Rotation ±15° (p=0.5)
- Random brightness/contrast (p=0.5)
- Gaussian noise (p=0.3)
- Color jitter (p=0.3)
- *Target:* bring all classes to ≥80% of the largest class count.

6.2 Model Training (`train_model.py`)

Base Model: `yolo11s-cls.pt` (YOLO11 Small Classification — latest architecture)

Training Hyperparameters

Parameter	Value	Reason
epochs	100	Max cap with early stopping
patience	20	Stop if no improvement for 20 epochs
batch	8	Safe for 4GB VRAM (GTX 1650)
imgsz	224×224	Standard ImageNet input size
optimizer	AdamW	Best for vision fine-tuning
lr0	0.002	Initial learning rate
lrf	0.01	Final LR fraction
weight_decay	0.0005	L2 regularization
warmup_epochs	5	Gradual LR warmup
cos_lr	True	Cosine annealing schedule
dropout	0.2	20% dropout prevents overfitting
amp	True (CUDA)	Mixed precision (FP16) → 2× faster

Device: Auto-detects CUDA; falls back to CPU.

Success Thresholds:

- ≥85% Top-1 Accuracy → Excellent
- ≥80% Top-1 Accuracy → Good

6.3 ONNX Export (`export_onnx.py`)

Converts `best.pt` → `waste_classifier.onnx` for CPU inference in Node.js:

```
best.pt (PyTorch) → ONNX (opset 12, simplified, FP32, static shape)
↓
server/models/waste_classifier.onnx (~21 MB)
server/models/model_info.json (class names, bin mapping, input shape)
```

Model spec: Input `[1, 3, 224, 224]` NCHW, FP32, ImageNet normalization

6.4 Inference Pipeline (`classifierService.js`)

```
imageBuffer
↓
preprocessImage() ← Sharp: resize 224×224, normalize ImageNet stats
↓
ort.Tensor('float32', data, [1,3,224,224])
↓
session.run() ← ONNX Runtime CPU inference
↓
rawLogits[8]
↓
softmax(logits) ← Numerically stable: subtract max before exp
↓
argmax → materialClass (e.g., "Plastic")
↓
BIN_MAP lookup → binCategory (e.g., "Recyclable")
↓
Aggregate bin-level scores from material probabilities
↓
Return { material, binCategory, confidence, scores, processingTimeMs }
```

Softmax (numerically stable):

```
const maxLogit = Math.max(...logits);
const exps = logits.map(x => Math.exp(x - maxLogit));
const sumExps = exps.reduce((a, b) => a + b, 0);
return exps.map(e => e / sumExps);
```

7. Database Schema

User

```
name, email, password (bcrypt), role (resident|admin|driver)
city, state, address, phone, plateNumber (drivers)
location: { type: Point, coordinates: [lng, lat] } ← 2dsphere indexed
ecoCredits (default: 100), earnings (default: 0)
```

Complaint

```
user (ref), bin (ref, optional), locationName, location {lat, lng}
city (enum: Thane|Nagpur|Mumbai|Pune|Nashik)
imageUrl, description, status (pending|assigned|on-the-way|resolved|rejected)
assignedDriver (ref), driverLocation {lat, lng}, routePath [[lat,lng]]
```

```
wasteSummary {category, material, volume, hazards, notes}
adminRemarks, isAuthorized (bool), rejectionReason, finalizedAt
eta, rewardAmount
```

Bin

```
location {lat, lng}, address, city
type (General Waste|Recycling|Organic|Hazardous)
status (empty|half-full|full|overflowing), fillLevel (0-100)
lastCollected, nextCollection
```

Scan

```
userId (ref), imageUrl
prediction { className (Recyclable|Compostable|Hazardous), confidence, allScores }
processingTimeMs, createdAt
Index: { userId: 1, createdAt: -1 }
```

Message

```
complaint (ref), sender (ref), recipient (ref), content, createdAt
```

Route / Truck

```
Basic logistics metadata for fleet and route tracking
```

8. API Reference

Authentication

Method	Endpoint	Access	Description
POST	`/api/auth/register`	Public	Register new user
POST	`/api/auth/login`	Public	Login, returns JWT
GET	`/api/auth/me`	Private	Get current user
PUT	`/api/auth/profile`	Private	Update profile

AI Analysis

Method	Endpoint	Access	Description
POST	`/api/analyze`	Public	Classify waste image (multipart)
GET	`/api/analyze/status`	Public	Check ONNX model health
GET	`/api/analyze/history`	Private	Paginated scan history

Complaints

Method	Endpoint	Access	Description
POST	`/api/complaints`	Resident	Submit new complaint
GET	`/api/complaints`	Private	Get city complaints
GET	`/api/complaints/my`	Resident	My complaints
PATCH	`/api/complaints/:id/accept`	Driver	Accept job
PATCH	`/api/complaints/:id/resolve`	Driver	Mark resolved
POST	`/api/complaints/:id/ping`	Admin	Broadcast to drivers
POST	`/api/complaints/:id/decline`	Admin	Decline complaint
PATCH	`/api/complaints/:id/finalize`	Admin	Authorize & disperse rewards
PATCH	`/api/complaints/sync-location`	Driver	Live GPS heartbeat

Bins / Trucks / Routes / Users

Method	Endpoint	Description
GET/POST	`/api/bins`	List/create bins
PUT/DELETE	`/api/bins/:id`	Update/delete bin
GET/POST	`/api/trucks`	Truck fleet management
GET/POST	`/api/routes`	Collection routes
GET	`/api/users`	All users (admin)
POST	`/api/logistics/optimize`	TSP route optimization

9. File Structure

```
clean-city-main/
├── client/ # React frontend
│ └── src/
│ ├── App.jsx # Router with role-based redirects
│ ├── context/AuthContext.jsx # JWT auth state (React Context)
│ ├── components/
│ │ ├── Layout.jsx # Sidebar + nav shell
│ │ ├── MapComponent.jsx # Google Maps wrapper
│ │ ├── PrivateRoute.jsx # Role-protected route guard
│ │ └── scanner/
│ │ ├── CameraFeed.jsx # Webcam with scanning UI overlay
│ │ ├── DropZone.jsx # Drag-and-drop image upload
│ │ └── ResultCard.jsx # AI result display with animations
│ └── pages/
│ ├── Login.jsx / Register.jsx / Settings.jsx
│ ├── admin/ # 6 admin pages
│ ├── resident/ # 5 resident pages
│ └── driver/ # 3 driver pages
│
├── server/ # Express.js backend
│ ├── server.js # Entry point, route mounting, socket init
│ ├── config/db.js # MongoDB connection
│ ├── middleware/authMiddleware.js # JWT protect/admin/driver guards
│ ├── models/ # Mongoose schemas
│ │ ├── userModel.js
│ │ ├── complaintModel.js
│ │ ├── binModel.js
│ │ ├── scanModel.js
│ │ ├── messageModel.js
│ │ ├── routeModel.js
│ │ ├── truckModel.js
│ │ ├── waste_classifier.onnx # Trained AI model (21MB)
│ │ └── model_info.json # Class names & bin mapping
│ ├── controllers/ # Business logic
│ │ ├── authController.js
│ │ ├── complaintController.js
│ │ ├── binController.js
│ │ ├── predictController.js
│ │ ├── routeController.js
│ │ ├── truckController.js
│ │ └── userController.js
│ ├── routes/ # Express routers
│ ├── services/socketManager.js # Socket.IO room management
│ ├── src/
│ │ ├── routes/analyzeRoutes.js # AI classify endpoint
│ │ ├── services/classifierService.js # ONNX inference engine
│ │ └── utils/tensorProcessor.js # Image preprocessing
```

```
│ └── logistics_service.py # FastAPI TSP optimizer
│
└── ml_pipeline/ # Python ML workflow
├── prepare_dataset.py # Merge & augment TrashBox + NewDataset
├── train_model.py # YOLO11s-cls training
├── export_onnx.py # Export to ONNX + generate metadata
├── test_model.py # Model validation script
└── requirements.txt
```

10. Key Algorithms Summary

Algorithm	Location	Purpose
Softmax	`classifierService.js`	Convert raw logits to probabilities
Nearest Neighbor TSP	`logistics_service.py`	Greedy route optimization
Haversine Formula	`logistics_service.py`	Accurate Earth-surface distance
Cosine Annealing LR	`train_model.py`	Smooth learning rate decay
Dropout (0.2)	`train_model.py`	Prevent overfitting during training
Albumentations Augmentation	`prepare_dataset.py`	Class balancing via synthetic data
bcrypt (salt=10)	`authController.js`	Secure password hashing
JWT HS256 (30d)	`authController.js`	Stateless session management
2dsphere Index	`userModel.js`	Geospatial MongoDB queries

11. Environment Variables

Server `.env`

```
PORT=5000
MONGO_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/cleancity
JWT_SECRET=<your_secret_key>
```

Client `.env`

```
VITE_API_URL=http://localhost:5000
VITE_GOOGLE_MAPS_API_KEY=<your_google_maps_key>
```

12. Getting Started

Step 1 — ML Pipeline (one-time)

```
cd ml_pipeline
pip install -r requirements.txt
python prepare_dataset.py # Merge & augment dataset
python train_model.py # Train YOLO11s-cls (needs GPU ideally)
python export_onnx.py # Export to server/models/waste_classifier.onnx
```

Step 2 — Backend

```
cd server
npm install
# Create .env with MONGO_URI and JWT_SECRET
npm run dev # Starts Express on :5000
# Optional: start logistics optimizer
pip install fastapi uvicorn numpy
python logistics_service.py # FastAPI on :8000
```

Step 3 — Frontend

```
cd client
npm install
# Create .env with VITE_API_URL and VITE_GOOGLE_MAPS_API_KEY
npm run dev # Vite dev server on :5173
```

13. Data Flow Diagram — Waste Scan

```
User opens WasteScanner.jsx
│
├─ Camera tab → CameraFeed.jsx → captures JPEG blob
└─ Upload tab → DropZone.jsx → selects file
↓
FormData { image: File } sent via Axios
POST http://localhost:5000/api/analyze
↓
Multer saves to server/uploads/scan-<timestamp>.jpg
↓
classifierService.classify(buffer)
├─ preprocessImage(): Sharp resize 224×224 + normalize
├─ ONNX inference: session.run()
├─ softmax() on 8 raw logits
```

```
├─ argmax → materialClass
└─ BIN_MAP[materialClass] → binCategory
↓
Scan saved to MongoDB (if authenticated)
↓
JSON response → ResultCard.jsx displays:
- Material name + bin category + emoji
- Animated confidence bar
- All 8 class score breakdown
- Disposal instruction text
- Processing time (ms)
```

Documentation generated from full codebase analysis — CleanCity AI v3